

Unit 5 — JavaScript Program Design

COMP266 — Introduction to Web Design

Script 1 — Dynamic Skills Section with Animated Progress Bars

1.1 — Data Structures

Global Array — skills

Scope: global (defined at top of script, accessible to all functions)
Type: Array of Skill objects

```
const skills = [ { name: 'JavaScript', level: 80, category: 'Frontend' }, { name: 'Python', level: 70, category: 'Backend' }, { name: 'CSS', level: 85, category: 'Frontend' }, { name: 'React', level: 75, category: 'Frontend' }, { name: 'Node.js', level: 60, category: 'Backend' }, { name: 'Git', level: 80, category: 'Tools' } ];
```

Class — Skill

Scope: global class definition
Purpose: encapsulates one skill's data and its HTML rendering logic

```
class Skill { constructor(name, level, category) // stores properties render() // returns an HTML string for the skill bar }
```

Local Variables

Variable	Scope	Type	Purpose
grouped	local — <code>groupByCategory()</code>	Object	Skills grouped by category key
container	local — <code>renderSkills()</code>	DOM Element	Target section in the page
observer	local — <code>initObserver()</code>	IntersectionObserver	Watches skills section visibility
bar	local — <code>animateBar()</code>	DOM Element	The individual bar element to animate

1.2 — Functions and Classes

Function / Method	Parameters	Returns	Description
<code>Skill.constructor()</code>	<code>name</code> , <code>level</code> , <code>category</code>	—	Initialises the three properties on the object
<code>Skill.render()</code>	—	String (HTML)	Returns a <code><div></code> string with the bar and label
<code>groupByCategory()</code>	<code>skillsArray</code>	Object	Groups skills by their category property
<code>renderSkills()</code>	—	—	Loops over grouped skills and injects HTML into the DOM
<code>animateBar()</code>	<code>barElement</code>	—	Sets bar width from 0 to data-level using a CSS transition

initObserver()	—	—	Creates an IntersectionObserver that fires animateBar() on scroll
----------------	---	---	---

1.3 — Pseudocode

```
// -- On page load -- CALL renderSkills() grouped = CALL groupByCategory(skills) FOR
each category in grouped: FOR each skill in grouped[category]: skillObj = new
Skill(skill.name, skill.level, skill.category) APPEND skillObj.render() to container
CALL initObserver() CREATE observer watching '.skills-section' // -- When skills
section enters viewport -- // -- callback fires: -- bars =
querySelectorAll('.skill-bar') FOR each bar in bars: CALL animateBar(bar) SET
bar.style.width = bar.dataset.level + '%' observer.unobserve() // animate only once
```

Script 2 — Multi-Criteria Project Filter with Live Text Search

2.1 — Data Structures

Global Variables

```
let currentFilter = 'all'; // currently active technology button let searchQuery =  
''; // text typed in the search bar
```

Local Variables

```
setFilter(tech): currentFilter = tech UPDATE active button style CALL applyFilters()  
// -- When user types in search bar -- FUNCTION setSearch(query): searchQuery =
```

Script 3 — Contact Form with Real-Time Validation

3.1 — Data Structures

Global Object — *formState*

Scope: global

Purpose: single source of truth for the validity state of all three fields

```
const formState = { nameValid: false, emailValid: false, messageValid: false };
```

Constants (*global*)

```
const EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]+$/; // validates email format
const MIN_NAME_LEN = 2; // minimum characters for name field
const MIN_MSG_LEN = 20; // minimum characters for message field
```

Local Variables

Variable	Scope	Type	Purpose
value	local — each validate fn	String	Trimmed value of the field being validated
isValid	local — each validate fn	Boolean	Result of the validation test
count	local — validateMessage()	Number	Current character count of message field

3.2 — Functions

Function	Triggered by	Description
validateName()	input event on name field	Checks length >= MIN_NAME_LEN. Updates formState.nameValid. Calls showFeedback() and updateSubmitButton()
validateEmail()	input event on email field	Tests EMAIL_REGEX against value. Updates formState.emailValid. Calls showFeedback() and updateSubmitButton()
validateMessage()	input event on message field	Checks length >= MIN_MSG_LEN. Updates character counter display. Updates formState.messageValid. Calls showFeedback() and updateSubmitButton()
showFeedback(field, valid, msg)	Called by validate fns	Displays a green or red message below the field. Uses text (not just colour) for accessibility
updateSubmitButton()	Called by validate fns	Enables submit button only if formState.nameValid AND emailValid AND messageValid are all true
initForm()	Page load (DOMContentLoaded)	Attaches input listeners to all three fields and submit listener to the form

3.3 — Pseudocode

```
// -- On page load -- CALL initForm() ADD input listener on nameField -> CALL
validateName() ADD input listener on emailField -> CALL validateEmail() ADD input
listener on messageField -> CALL validateMessage() // -- On each keystroke in name
field -- FUNCTION validateName(): value = nameField.value.trim() IF length(value) >=
MIN_NAME_LEN: formState.nameValid = true CALL showFeedback(nameField, true, 'Name
looks good') ELSE: formState.nameValid = false CALL showFeedback(nameField, false,
'Name must be at least 2 characters') CALL updateSubmitButton() // -- On each
keystroke in email field -- FUNCTION validateEmail(): value = emailField.value.trim()
IF EMAIL_REGEX.test(value): formState.emailValid = true CALL showFeedback(emailField,
true, 'Email is valid') ELSE: formState.emailValid = false CALL
showFeedback(emailField, false, 'Enter a valid email (e.g. name@example.com)') CALL
updateSubmitButton() // -- On each keystroke in message field -- FUNCTION
validateMessage(): value = messageField.value.trim() count = length(value) UPDATE
counter display: count + ' / ' + MIN_MSG_LEN + ' min characters' IF count >=
MIN_MSG_LEN: formState.messageValid = true CALL showFeedback(messageField, true,
'Message is long enough') ELSE: formState.messageValid = false CALL
showFeedback(messageField, false, 'Message too short') CALL updateSubmitButton() //
-- Update submit button state -- FUNCTION updateSubmitButton(): IF
formState.nameValid AND formState.emailValid AND formState.messageValid:
submitBtn.disabled = false ELSE: submitBtn.disabled = true
```